03-Aug

Gradient Descent

→ Repres. ∇ (nabla)

→ Iterative optimization algorithm used in ML and DL to minimize the cost function $J(\theta)$ by finding optimal parameters (weights & biases) of Model

- $\nabla f(x,y) = \left\langle \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right\rangle$
- $\nabla f(x,y,z) = \left\langle \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right\rangle$

Q $f(x,y) = 2x^2y - 4y^2$

$$\frac{\partial f}{\partial y} = 2x^2 - 8y$$

$$\frac{\partial f}{\partial x} = 4xy$$

$$\nabla f(x,y) = \langle 4xy, 2x^2 - 8y \rangle$$

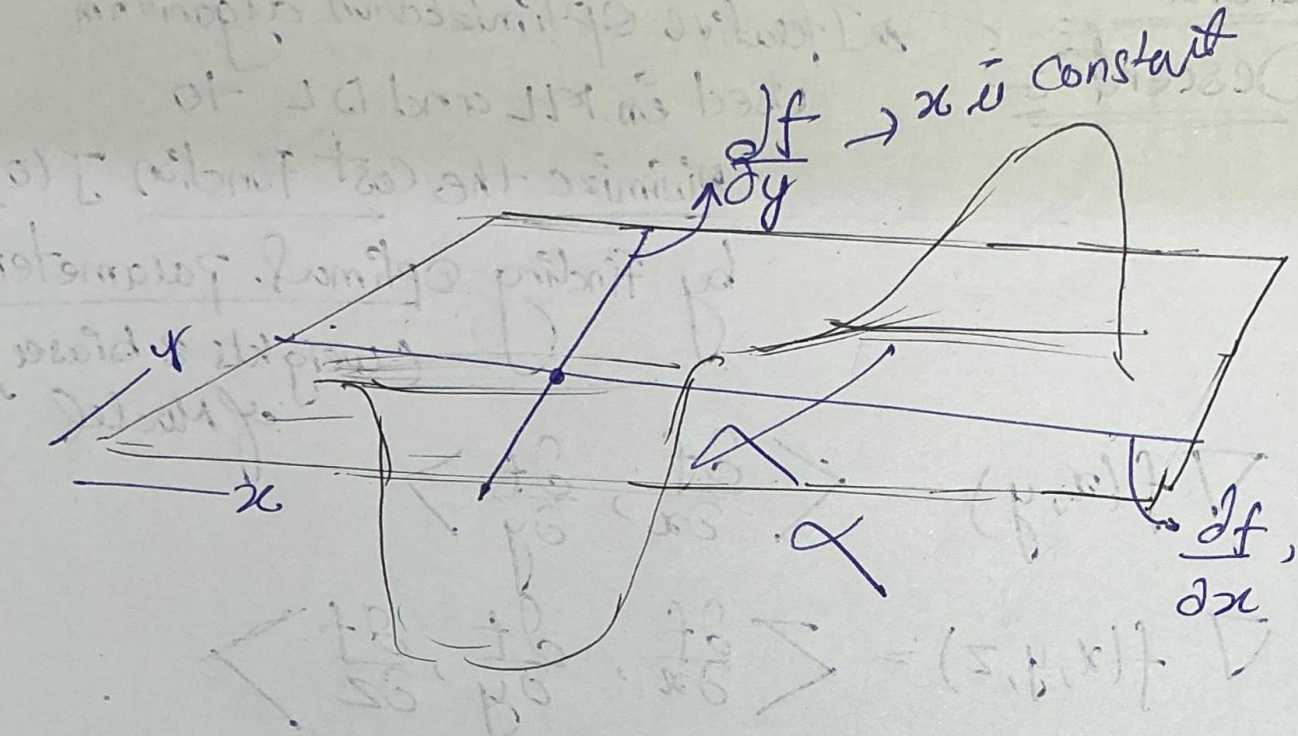
Gradient Vector at point (1,2),

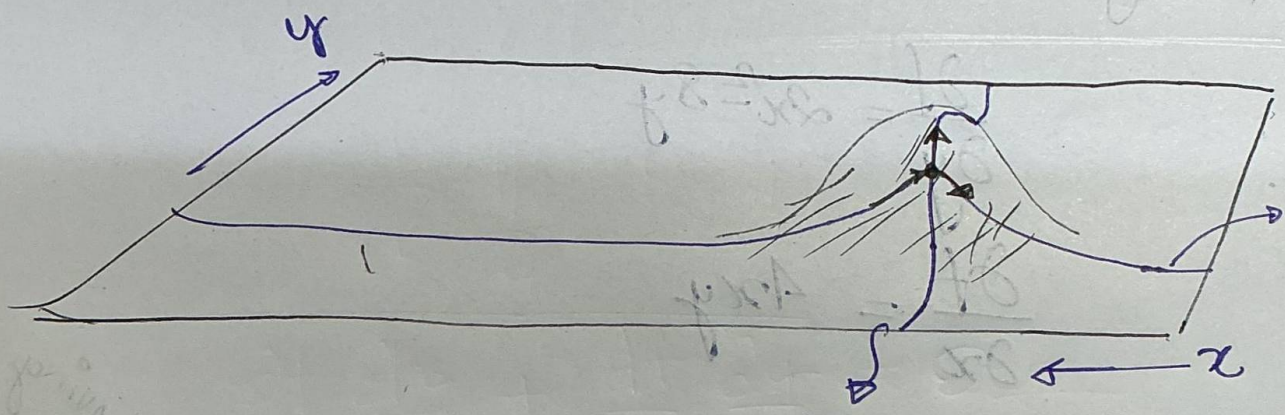
$x=1, y=2$

$$\nabla f(1,2) = \langle 8, -12 \rangle$$

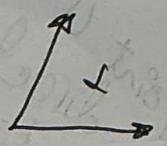
$$= \underline{8\vec{i} + 14\vec{j}}$$

At the point (1,2) moving in the direction of this vector will take uphill the fastest.

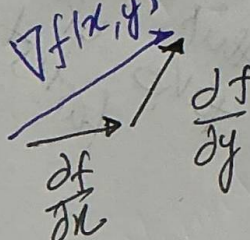




$\frac{\partial f}{\partial y}$ (x ⓘ Constant)



Slope

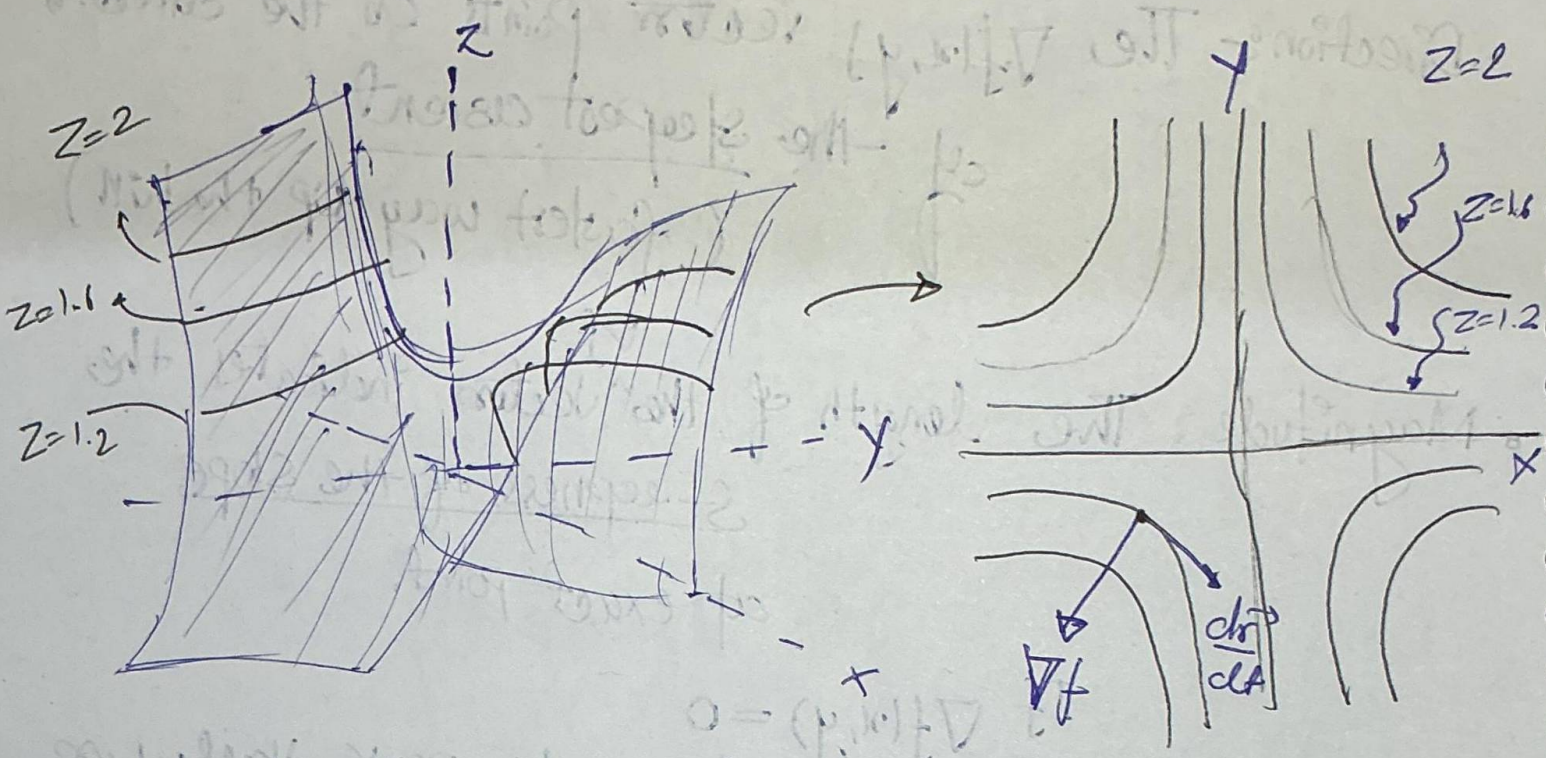


$$\nabla f(x,y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

- Direction:- The $\nabla f(x,y)$ vector points in the direction of the steepest ascent.
  (fastest way up the hill)
  ∇f
- Magnitude:- The length of the vector indicates the steepness of the slope at exact point.

If $\nabla f(x,y) = 0$
↳ flate peak, valley or saddle point

- Perpendicularity:-
  The gradient vector at any point is always perpendicular to the level curves (contour lines) of the function.



Suppose along a curve $\vec{r}(t) = x(t)\hat{i} + y(t)\hat{j}$

Then $f(x(t), y(t)) = C$

$\frac{d}{dt} f(x(t), y(t)) = \frac{d}{dt}(C)$

$\frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{df}{\partial y}\frac{\partial y}{\partial t} = 0$

$\left(\frac{\partial f}{\partial x}\hat{i} + \frac{\partial f}{\partial y}\hat{j}\right) \cdot \left(\frac{dx}{dt}\hat{i} + \frac{dy}{dt}\hat{j}\right) = 0$

Tangent.

Normal

$$\nabla f \cdot \frac{d\vec{r}}{dt} = 0$$

So, If we change our position vector by tiny fraction ($dx_1, dx_2, \ldots dx_n$) the total change is divergence

$$df = \frac{\partial f}{\partial x_1} dx_1 + \frac{\partial f}{\partial x_2} dx_2 + \ldots + \frac{\partial f}{\partial x_n} dx_n$$

$$df = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \cdot \begin{bmatrix} dx_1 \\ dx_2 \\ dx_n \end{bmatrix}$$

$$df = \nabla f \cdot d\vec{v}$$

∇f → gradient, $d\vec{v}$ → Tangent vector.

$$df = \| \nabla f \| \, \| d\vec{v} \| \cos(\theta)$$

For the largest possible increase in our function (df) → Max $\cos(\theta)$.

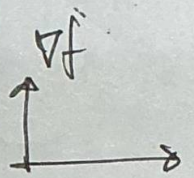
So, $\cos\theta = 1 \Rightarrow \boxed{\theta = 0^\circ}$

Thus displacement vector ($d\vec{v}$) must point in exact same direction as gradient vector ∇f.

Therefore the gradient vector directly in the direction of steepest ascent

NOTE:-

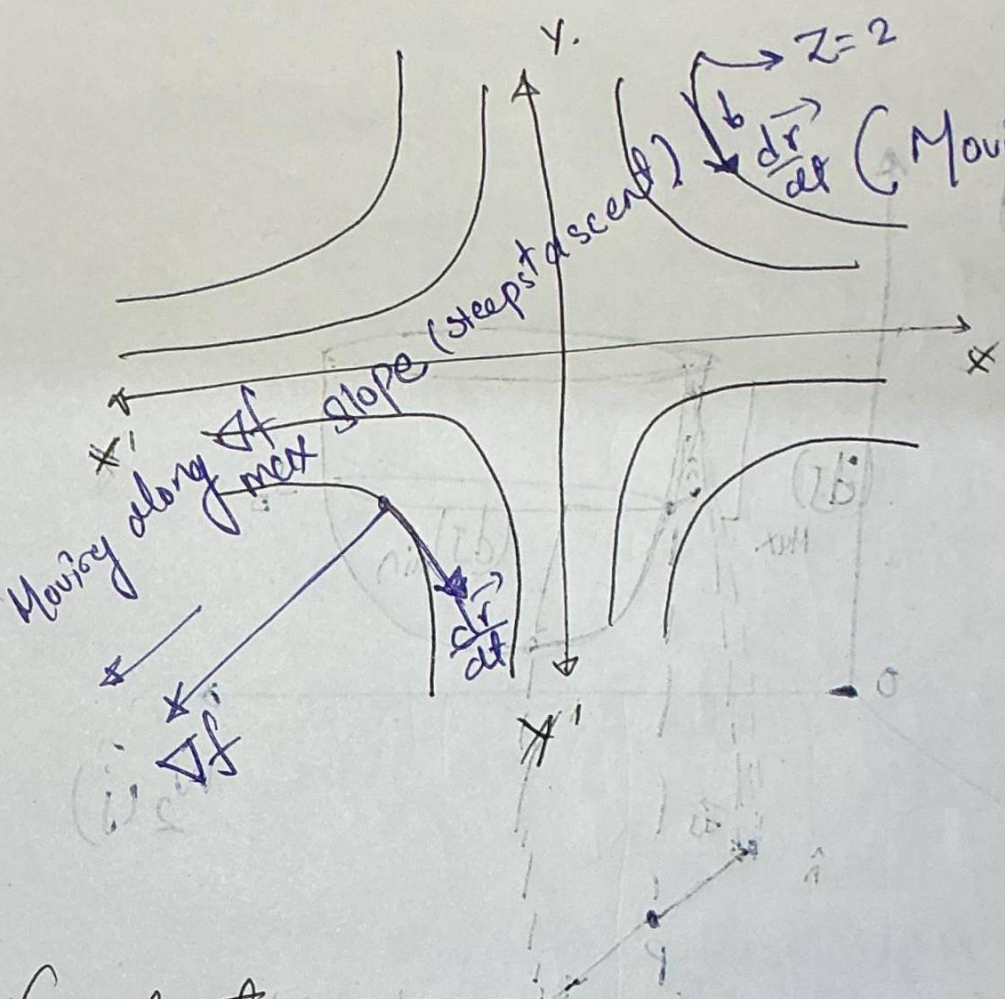
1) Smallest Magnitude when $\theta = \pi/2$



i.e $\frac{d\vec{r}}{dt}$ is direction of minimum slope

2) Largest magnitude when $\theta = 0^\circ$

i.e. ∇f is direction of maximum slope

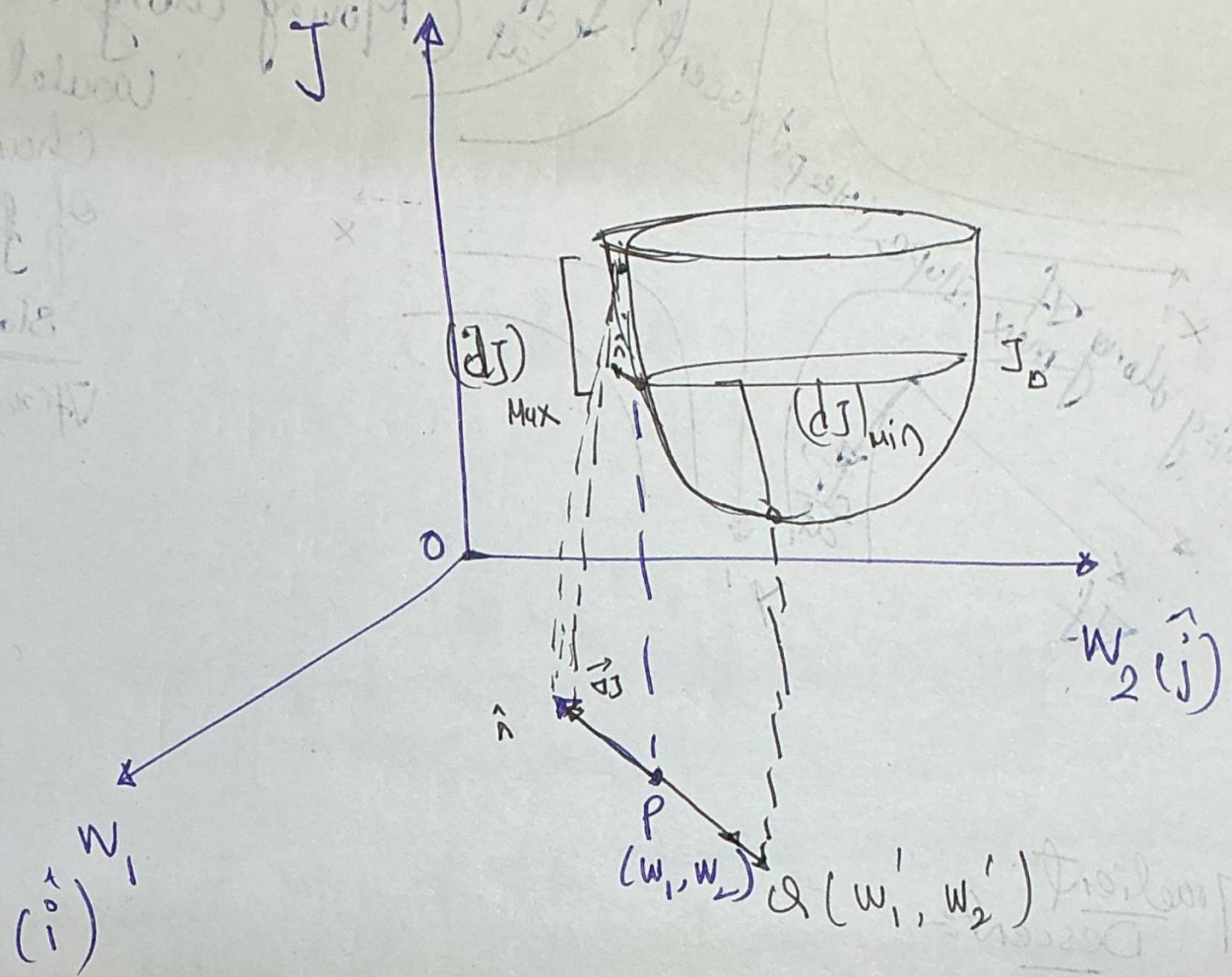


$\nabla f(x,y)$

Gradient Descent:-

$J = f(w_1, w_2, \dots w_n)$

Suppose $J = f(w_1, w_2)$ → To minimus the Cost function J we have to find apropiate value of w_1 and w_2.



Now we know about directional derivative.

$$(df = \vec{\nabla} f \cdot d\vec{v})$$

$$dJ = \vec{\nabla} J \cdot \hat{n} \quad -(i)$$

$$dJ = \|\nabla J\| \|\hat{n}\| \cos\theta \quad \text{if } \theta = 0^\circ \Rightarrow \cos\theta \Rightarrow \text{Max}$$

$$\vec{dJ}\,(\text{Max})$$

$$\vec{\nabla J} = \frac{\partial J}{\partial w_1}\hat{i} + \frac{\partial J}{\partial w_2}\hat{j}$$

$\vec{r} = |\vec{r}|\hat{r}$

$$\vec{OQ} = \vec{OP} + \vec{PQ}$$

$$\therefore \vec{OP} = w_1\hat{i} + w_2\hat{j}$$

$$\vec{OQ} = (w_1\hat{i} + w_2\hat{j}) + \vec{PQ}$$

$$\therefore \vec{PQ} = \left(\frac{-\vec{\nabla}J}{|\nabla J|}\right) \cdot |\vec{PQ}|$$

unit vector along $\vec{PQ}$

$$\therefore \vec{OQ} = w_1'\hat{i} + w_2'\hat{j}$$

as PQ is small $\rightarrow \delta$

so,

$$(w_1'\hat{i} + w_2'\hat{j}) = (w_1\hat{i} + w_2\hat{j}) - \frac{\vec{\nabla}J}{|\nabla J|}\delta$$

$$(w_1'\hat{i} + w_2'\hat{j}) = (w_1\hat{i} + w_2\hat{j}) - \left(\frac{\delta}{|\nabla J|}\right)\left(\frac{\partial J}{\partial w_1}\hat{i} + \frac{\partial J}{\partial w_2}\hat{j}\right)$$

α (learning rate)

$$(w_1'\hat{i} + w_2'\hat{j}) = (w_1\hat{i} + w_2\hat{j}) - \alpha\left(\frac{\partial J}{\partial w_1}\hat{i} + \frac{\partial J}{\partial w_2}\hat{j}\right)$$

so,

- $w_1' = w_1 - \alpha\frac{\partial J}{\partial w_1}$
- $w_2' = w_2 - \alpha\frac{\partial J}{\partial w_2}$

or

$$w := w - \alpha\frac{\partial J}{\partial w}$$

Types of GDs

① Batch Gradient Descent

Calculates the gradient of the loss function across entire training dataset before making a single parameter update.

- If dataset contains N samples, it computes the avg gradient across all N samples to complete exactly one step.

Contour plot.

- The trajectory is exceptionally smooth, stable, and direct. Because it uses all data, it always knows the exact

Pros:- 1) Highly Stable Convergence
2) guaranteed to Convege to global min for Convex function

Cons:- 1) Computationally expensive and slow for large dataset
2) It requires loading the entire dataset into computer memory (RAM)
↓
Impossible for Massive data.

Psedocode:-

Inputs:
X: Training features (Matrix size N x M)
Y: True label (vector size (N,)
α: learning rate or stepsize
Max-epochs: Total No. of passes over data
tol: Stopping Condition for early exist.

Output:
theta: Optimized Parameters (weights & bias)

Procedure:

1. Initialize theta randomly or with zero.
2. For epoch = 1 To Max_epochs Do:
   a) Calculate Predictions:
      Y_Pred = Model_func (X, theta)
   b) Compute loss (e.g MSE)
      cussef_loss = Compute_loss (y, y_Pred)
   c.) Compute True Gradient across ALL N samples:

$$gradient = \left(\frac{1}{N}\right) * X^{T} . * (y\_Pred - y)$$

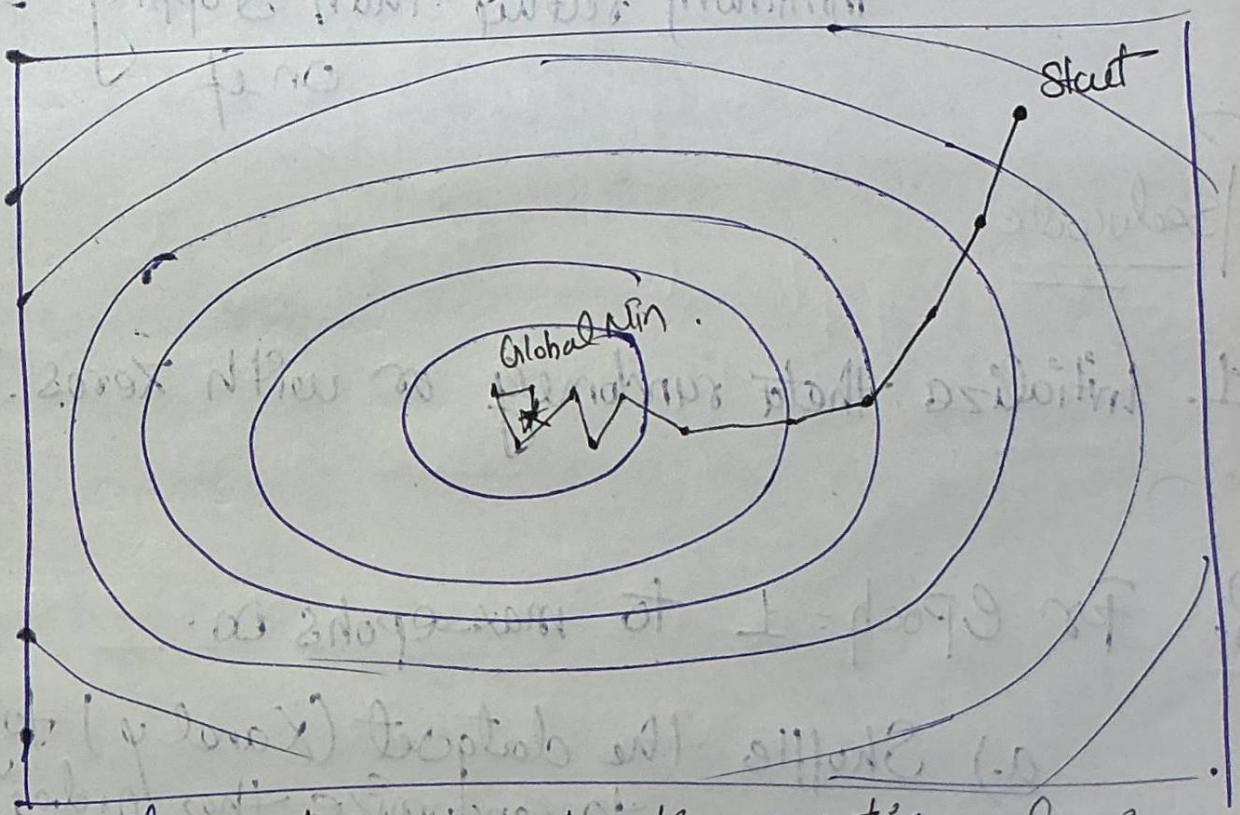
   d.) Check Convegence
      If mgnitude (gradie) < tot Then:
         Break (Stop traing, min fare)
   e. Update parameter (Take one step downhill)
      theta = theta - (α * gradient)
3. Return theta.

(2) Stochastic Gradient Descent (SGD)

Updates Model parameters sequentially after analyzing <u>just one random training sample</u>

- for every individual row of data, gradient is calculated and parameter step is taken immediately.

For N sample → N updates in parameters.



The path looks highly chaotic, erratic and noisy.

Because individual data points have unique noise, step will often jump sideways or even uphill

However, the overall avg. trend still migrates downward.

Pros:- 1) Extremely fast per step and require almost zero memory

2) The chaotic jumping can actually help algo bounce out of bad local minima or flat plateaus.

Cons:- 1) It never truly settle down

2) Even when it finds the absolute bottom, the inherent noise causes it to perpetually bounce around the minimum rather than stopping exactly on it

Pseudocode.

1. Initialize theta randomly or with zeros.

2. For epoch = 1 to max_epochs do.

   a.) Shuffle the dataset (X and y) together to randomize the order

b. For each individual Sample i from 1 To N Do:

i. Calculate Prediction for just to this One Sample

$$y\_pred_i = Model\ function(X[i], theta)$$

ii: Compute the gradient for just this one Sample:

$$gradient\_i = X[i] * (y\_pred\_i - y[i])$$

iii) update Parameters immediately

$$theta = theta - (\alpha * gradient\_i)$$

3. RETURN theta.

(3.) Mini-Batch Gradient Descent. (Industry Standard)

Used in Modern deep learning and neural networks.

↳ It splits the data into small, fixed-size groups called "batches"

- Typically batch sizes range from 32 to 512 samples.

  also Computes the avg gradient from batch

  ↓

  Updates the parameters

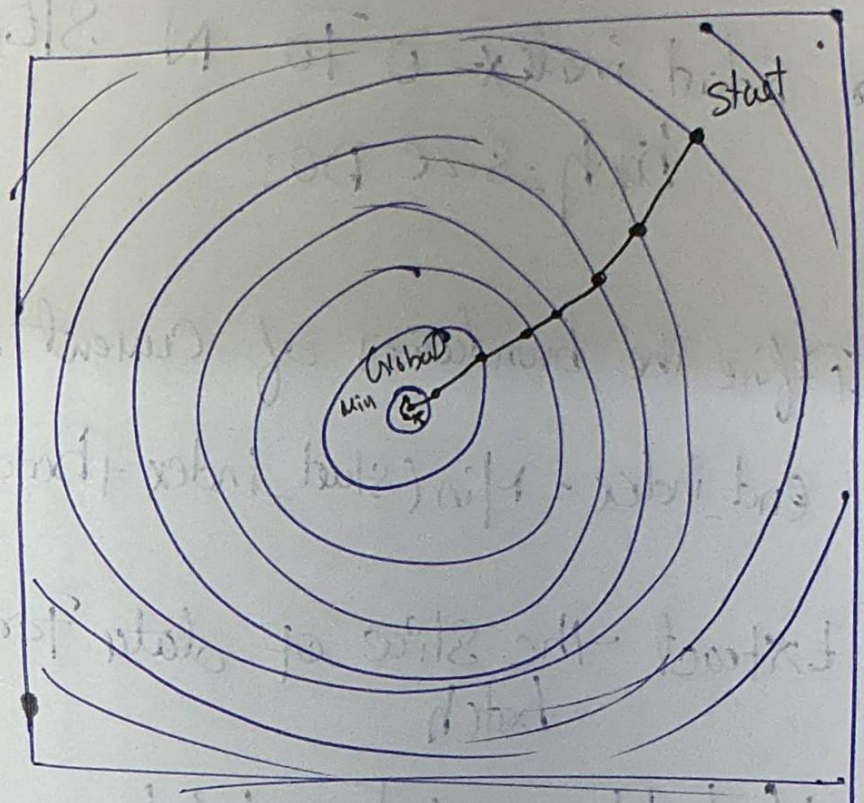
  ↓

  Moves to next batch.

Pros:- 1) Highly optimized for Hardware

2) GPUs are best to calculate matrix operation Parallely.

Cons:- 1) Introduces a new hyperparameter that must be tunned (Batch size) Manualy.



It strikes a perfect balance. The trajectory is relatively smoth and efficient, but retains a tiny bit of healthy oscillation to escape shallow structral traps.

Pseudocode :-

1. Initialize theta
2. For epoch = 1 to Max epoch Do:
   a.) Shuffle the dataset (X and y)

b) For start_index = 0 TO N STEP batch_size DO:

i) Define the boundaries of current mini_batch

- end_index = Min (start_index + batch_size, N)

ii) Extract the slice of data for this batch

X_batch = X [start_index : end_index]

Y_batch = Y [start_index : end_index]

K = No. of elements in this batch (end_index − start_index)

iii) Calculate Predictions for this batch

Y_Predict_batch = Model_fun (X_batch, theta)

iv) Compute the avg Grad Gradient across this batch:

gradient_batch = $\left(\frac{1}{K}\right) * (X_{batch})^T * (y\_Pred - y\_bat$

N) Update parms

theta = theta − α * gradient batch

3. RETURN theta.

update (N/batch_size) per epoch.